

Comparison of the System Development Life Cycle and Prototype Model for Software Engineering

Maryani¹, Harjanto Prabowo², Ford Lumban Gaol³, Ahmad Nizar Hidayanto⁴

¹Information Systems Department School of Information System Doctor of Computer Science Bina Nusantara University & Jakarta, Indonesia 11480

²Management Department BINUS Business School – Undergraduate Program Bina Nusantara University & Jakarta, Indonesia 11480

³Computer Science Department BINUS Graduate Program - Doctor of Computer Science Bina Nusantara University & Jakarta, Indonesia 11480

⁴Faculty of Computer Science Universitas Indonesia, & Depok, Indonesia 16424

Abstract— The concept of system lifecycle models was born, emphasizing the importance of adhering to them. Some methodical technique to developing a new or improved system. Several models were suggested, including the waterfall model, the V-shaped model, the iterative model, the spiral model, and the Bing bang model are all examples of models., etc. The comparative analysis is the emphasis of this paper. These Software Development Life Cycle Models were examined. Organizations' operations are expanding, and the need to automate certain processes has grown. As a result, it was thought that standard and structural technique, or It is suggested that a methodology be established in the industry so that the transition from a manual to an automated system may be made as smoothly as possible.

Keywords—SDLC Model, Comparative, methodology, developing model

I. INTRODUCTION

There are multiple models for the process, each of which describes a different approach to the various activities that take place during it. Each The software development process is broken down into logical steps. phases can aid in the effective organization of a software development firm its resources. efforts to provide a product of software with the desired capabilities within a set time and financial constraints. Rodriguez Martinez et al. focus on life cycle framework construction of Life cycle processes are represented in detail by extensive software models, and they present the findings of a comparative study of the software development life cycle that enables a reasonable an explanation of its development in terms of common, distinctive, and aspects that are distinctive and specifications with thoroughness and qualities of agility.

An investigation of the notion technique as well as a meta-model of the software process life cycle are employed to achieve this goal. The essential ideas and a comparison of software development models were described in the study by Jovanovich D. et al. The first half describes the development model, and the second part explains how to put one of the paradigms for software development into practice. Finally, there is the issue of identifying the best creation of software methodology for PC application development. Present a framework that can be used to compare alternative models of the life cycle as a tool for software engineering academics to discuss the possible consequences of life cycle models and to aid software practitioners in deciding which life cycle models are acceptable to utilize in a particular project or application area. A Comparative Analysis of Software Process Improvement Model was provided by B. Sharma and colleagues. The importance of improving software processes is widely acknowledged. The Software Development Life Cycle Model is a collection of tasks that must be completed for software to be developed. along having a connection of ordering between them, performed in a way that will create the desired result by satisfying the ordering relationship. The SDLC paradigm is a method for developing software is a simplified version of the envelopment procedure. The purpose of software development is to provide high-quality software. [1]. As a result, the development process is the set of actions that leads to the creation of software. The software development life cycle is divided into several stages. activities and determines how are these actions planned out within the overall effort put towards software creation.

In reaction to the conventional wisdom, approach to the creation of software, a new ultra-lightweight methodology has emerged. A high percentage of attempts to build software have no processes and are perhaps the best way to describe it is chaos "code and repair" activities. The lightweight SDLC technique is a compromise between no process and too many processes. Twelve types of lightweight Waterfall, V-shaped, iterative, spiral, and Bing bang models are examples of SDLC approaches, etc. Involved activities A Model of the Software Development Life Cycle The following activities are involved in software problem solving: 1. Recognize the problem 2. Create a plan for resolving it 3. Writing the Code for the Proposed Solution 4. Putting the Program to the Test Each task in a huge system might be exceedingly complicated, necessitating the use of techniques and processes to complete it efficiently and correctly. Furthermore, one of the primary actions may be so enormous that it is impossible to execute them all in one go and must be split down into smaller chunks. A complex software system, for example, is always split down into many independent Starting with a very high-level design, there are several design phases that just specifies the component. The following are the basic tasks or phases that must be carried out to construct a kind of software: 1. the Requirements of the System, 2. Development of a system, 3. Software development and 4. System Validation.

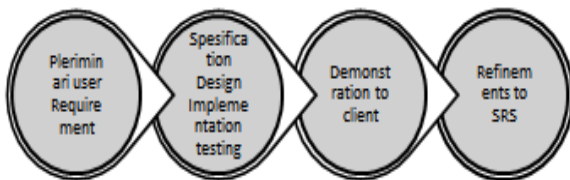


Fig. 1 Prototyping Model

The feasibility of the project is examined at this phase, as well as a business proposal presented using a broad-brush outline for the project, as well as some budget projections. The development activities begin after the business idea has been approved. or the contract is awarded, commencing with the requirement analysis.

II. COMPARATIVE ANALYSIS

There are many approaches used for software development in building projects that are completed. From the literature study, the researchers tried to multiply the various existing methods and choose one that is in accordance with the retail business or clothing augmented reality. Among these methods, the Life Cycle of Software are:

1. Waterfall Model

The oldest is a waterfall. and easiest structured SDLC methodology, i.e., finish one phase, then move continue the stages. Each stage builds on the previous one's content It is self-contained and has its own project plan. The waterfall paradigm is simple to grasp and manage. Early delays, on the other hand can throw the entire project schedule off. Because there isn't much space for adjustment once the phase is over, the issue will not be resolved until the maintenance phase has been reached. When there is a requirement for flexibility or if the project is long-term and sustainable, this paradigm does not work effectively [2][3].

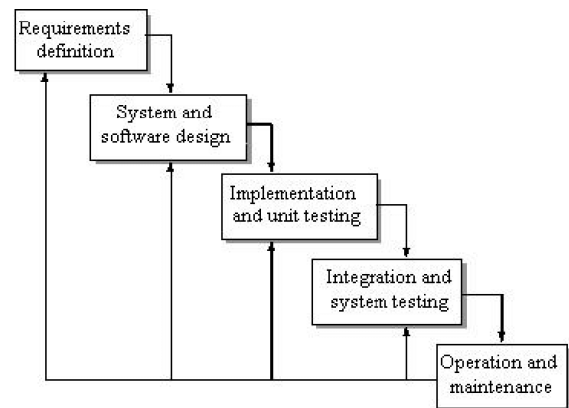


Fig. 2.2. Waterfall Model [2].

2. Model with a V-shape

A V-Model is an SDLC model in which each phase's execution is carried out in a V-shaped pattern. The V-Model is a variation on the waterfall model that is based on the relationships between testing stages and phases. This means that there is a test phase that is directly tied to each component of a cycle.

Like the waterfall model, this model applies a very high level of discipline where the next phase cannot be started before the previous phase is completed. In the V-Model, the testing stages in one cycle are carried out in parallel. So, in the V-Model, there is a contract is awarded or a business proposal is accepted. both sides of this V-Model [2][4].

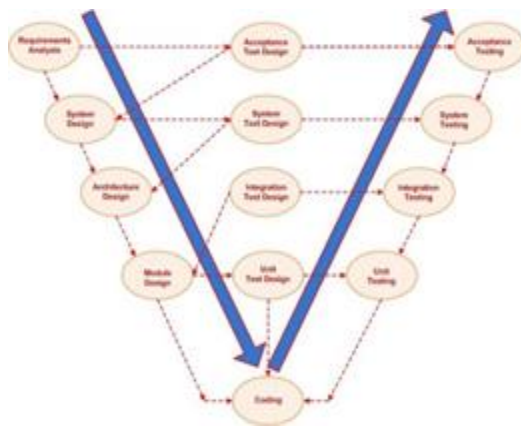


Fig. 2.3. Model with a V-shape [2].

3. Model of iteration

The iterative paradigm begins with a rudimentary implementation of a subset of software requirements and continues to update versions until the complete system is ready to be implemented. In design changes are made with each iteration, and new functional capabilities are added. The primary the concept behind it strategy is to develop the system in small increments through iterative cycles (incremental).[2]

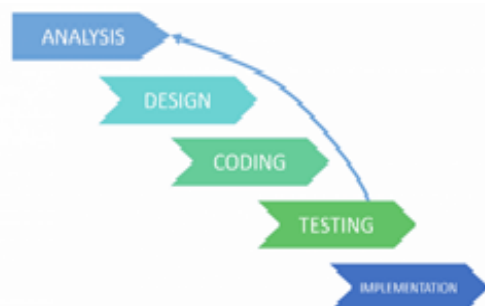


Fig. 2.4. Iterative Model [2]

The iterative and incremental SDLC paradigm has several advantages: allows for the execution and development of functions that need to be completed quickly in the early stages of work, early and regular results are acquired, and parallel development is conceivable., measurable advancement, fewer resources to adapt to changing scope and requirements, Perform tests and patches on defined iterations with ease; risks are identified and managed during iterations; each iteration is a reasonable milestone, risk management is simplified because the high-risk portion is completed first, with each upgrade, operational products are delivered, each increment's problems, challenges, and risks can be leveraged and applied to the next increment, Improved risk assessment, adapts to changing requirements, Initial operating times tend to be short, appropriate for large and important projects, early in the cycle, software is generated, allowing for customer comments and assessment.

The negative aspects of the SDLC model that is iterative may necessitate additional resources, despite its lower cost of change, it is unsuitable in response to shifting needs. More management attention is needed: If all requirements are met, system architecture or design difficulties may develop are not collected early in the life cycle, lining upgrades may necessitate a full system definition, Insufficient for smaller projects added management complexities, the risks to the final project may be less well known, the use of Risk analysis necessitates the use of professional resources, and the risk analysis phase is crucial to project success.

4. Model with Spirals

One of the most popular models is the spiral model. adaptable SDLC approaches its cues because of the iterative and spiral models; the project goes through four iterative phases in a "spiral" to completion, allowing for a few iterations. This design enables us to build extremely personalized products, when early on in the process, consumer feedback may be incorporated. However, there is a danger involved is creating a never-ending spiral for ongoing projects [2].



Fig. 2.5. Spiral Model [2]

The advantages:

New prototypes are obtained all the time. Reuse capabilities, better productivity, it has a systematic step-by-step approach and the elimination of errors at an early stage. The disadvantages: Improper cost and time estimates, Cost of high-risk analysis on large projects, Complex.

5. Big Bang Model

Working on software development projects is not a distinct approach in the Big Bang SDLC model. With a minimal amount of money and basic resources, a project can be begun. There was little preparation involved in the Big Bang's functioning model, and no formal procedure was used. This model is used when the buyer is unsure of his desires and his requirements have not been thoroughly examined. Because Big Bang is a dangerous situation to be in methodology, For big or difficult tasks, it is not recommended. The emphasis is still on producing code and delivering the finished result. This paradigm is significantly risky than other models since it lacks preparation and analysis [5].

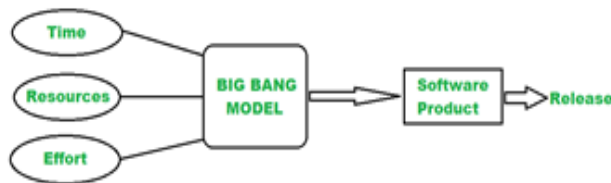


Fig. 2.6. Bing Bang Model [5]

6. Modelling Agility

The Modelling Agility, it separates the product into cycles, generates a functional product quickly, and is considered a very useful development approach.

This approach produces a succession of releases, each with tiny differences from the last. Each iteration of the product is put to the test. In this model, customers, developers, and testers cooperate across projects, which encourages engagement. Because this strategy is so reliant on client interaction, projects might move in the wrong direction if the customer is unclear of their objectives [6][7].



Fig. 2.7. Modelling Agility [6]

The benefits of modelling agility: System development is made faster, increases user satisfaction, and reduces the risk of failure of software implementation in non-technical terms. If at the time of system development there is a failure, the material loss is relatively small. The modelling agility Disadvantages: If you are not ready to face some changes, the results will be less than optimal. If the team is difficult to adjust to the fast work time, the initial plan can change, at any time, the schedule is uncertain, and if the team cannot communicate well, there will be setbacks.

7. Prototype Model

The evolutionary process is represented by the prototype model. The Evolutionary Process Paradigm is an iterative model that is used to generate ever-more-complete software versions. A rapid design step is included in the prototype model, in which the developer must design the software quickly after gathering requirements [8][9].

Clients can truly feel the system, i.e., feedback design in advance. Advantages: Early visibility, higher output, and lower costs. Negative aspects include the possibility of insufficient or incomplete systems. There is a lack of flexibility. Because management is difficult, it is not ideal for huge systems.

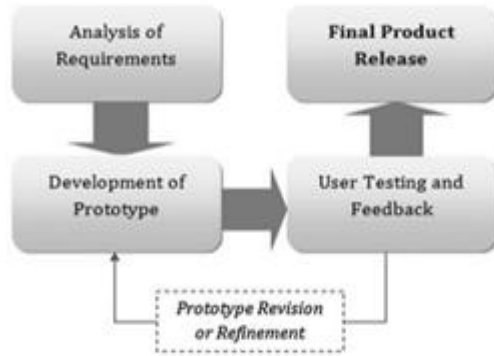


Fig. 2.8. Prototype Model [8]

8. Scrum Model

Scrum is a framework in which parties can find solutions to complex problems and, at the same time, create products that have the highest possible value in a productive and creative way. Scrum is designed to speed up development time, unify individual and organizational goals, create a performance-driven culture, assist the generation of shareholder value, and provide stable and constant communication for every level of performance, and enhance individual development and quality of life. [10] describes the characteristics of Scrum, namely: Simple, easy to understand, difficult to master. The owner of the product, SCRUM master, and team of developers/Scrum masters are the first three roles in the SCRUM technique. The product owner oversees dealing with stakeholders, while the Scrum Master oversees internal matters, and the development team is in charge of technical project work and more in-depth conversations.

Many multinational companies internalize or adopt this technique as their standard of work because, in general, the scrum technique has succeeded in making several companies increase turnover because of this proven system. The Scrum technique makes work more organized and more detailed. However, everything has its pluses and minuses, as well as this Scrum technique. Scrum makes work neater, but this technique is not suitable to be applied to service companies that need fast deadlines. Scrum causes work to take longer than expected. However, everything can be done if you already have careful preparation from the start [10][11].

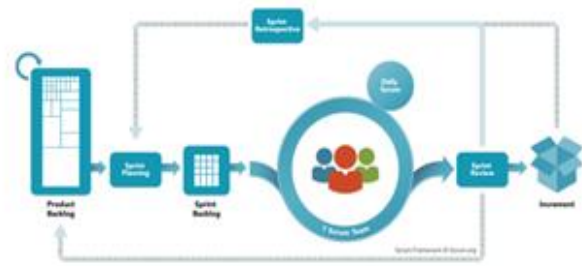


Fig. 2.9.. Scrum Model [10]

9. Augmented-reality interactive technology (ARIT)

ARIT is a more advanced image interaction technology that creates and manipulates augmented-reality interactive technology reception models based on website parameters. In augmented-reality interactive technology (ARIT), images of products or settings are used to simulate the user's actual experience with the object or location. Virtual reality, for instance, entirely immerses web users in a synthetic environment while preventing them from perceiving the physical world around them. ARIT, on the other hand, allows online shoppers to focus on the product's characteristics, iterate and examine items from several sources' perspectives, as well as observe objects in a variety of virtual colours models that are supposed to resemble the client. ARIT can give product data that is very similar to that collected during in-person product inspections, reducing product risk. In a nutshell, ARIT allows users to view three-dimensional virtual items superimposed on real-world settings via the internet. ARIT has the following characteristics, according to past research:

- 1) match actual and virtual items and mix real and virtual content
- 2) real-time interactive and visible
- 3) Real-time registration of virtual content

It can instantly reflect the online consumer's facial expressions, images, and environment on the computer screen, and can also integrate virtual objects directly onto the online consumer's figure, achieving the same effect as if they were trying it, according to the ARIT characteristics above and through the ARIT webcam function.

The online consumer, on the other hand, has influence over the order of serving in ARIT. It is capable of simultaneously and without latency replicating the consumer's bodily motions on the screen. As a result, online shoppers can examine product content according to their own preferences and in any order, they want, such as by clothing type, size, or colour category, without being restricted. In brief, ARIT shapes the online buying experience from a first-person perspective, like how online clothing fitting works. [12].

ARIT advantages: the main advantage of ARIT is that it is easier and cheaper to develop and can be widely implemented in various media. as an application on a smartphone, game console, in a product package, or even print media such as books, magazines, or newspapers.

One of the most difficult aspects of designing Augmented Reality applications is tracking the user's position of view, which is one of ARIT's drawbacks. The application must know where the user is looking in the actual world to determine which point of view the virtual image is appealing from.

10. ADDIE (Analysis, Design, Development, Implementation and Evaluation)

As the name implies, the ADDIE Development Research Model consists of five stages or development analysis, design, development or production, implementation or delivery, and assessments are the different steps. In 1996, Dick and Carry came up with the ADDIE concept to construct a learning system [13]. The purpose of the ADDIE development research model is to produce or develop an empirically tested product. To produce these new and tested products, it is necessary to have documented and measurable stages of activities at all stages of development and manufacture.



Fig. 2.10. ADDIE Model [13]

11. RAD Model

RAD is a way for developing information systems that takes a short amount of time to complete. Normal information system development takes at least 180 days, however information system work can be accomplished in 30–90 days employing the RAD technique. Users can participate in the whole system development process by acting as decision makers at each step of development when RAD is applied. Because the system built may match the wishes of the users, RAD also develops a system rapidly, decreasing the time required for rebuilding after the implementation phase [14].

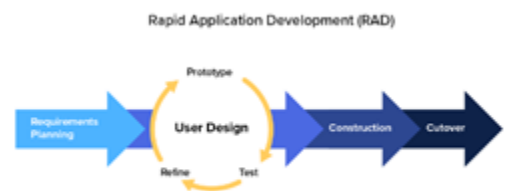


Fig. 2.11. RAD Model [14]

The following are some of the benefits of employing the RAD method: Buying a new system is less expensive than designing your own, the delivery process is simplified. This is due to the fact that the manufacturing process necessitates the usage of additional script elements, Simple to observe because it employs a prototype model, allowing users to gain a better understanding of the system under development, More adaptable because developers can work on the redesign simultaneously, Because it employs a wizard, it can decrease the need to write difficult code, Because users are a part of the overall team, their involvement is growing, Able to use CASE tools to reduce errors. The following are some of the drawbacks of employing the RAD method: a purchase may not always save money as compared to constructing your own, Separate charges for supporting equipment such as software and hardware are required.

12. DevOps Model

DevOps is a set of concepts of culture, techniques, as well as technology that improve a company's capacity to deliver high-volume apps and services, allowing it to build and enhance products in less time than conventional approaches to software development and infrastructure management. businesses because of this, they will be able to give their clients better service and compete more successfully in the industry, this quickness [15].

It improves products in less time than Traditional software development and infrastructure management procedures are used by enterprises. Because of this speed, businesses can better serve their customers and compete in the market [10].

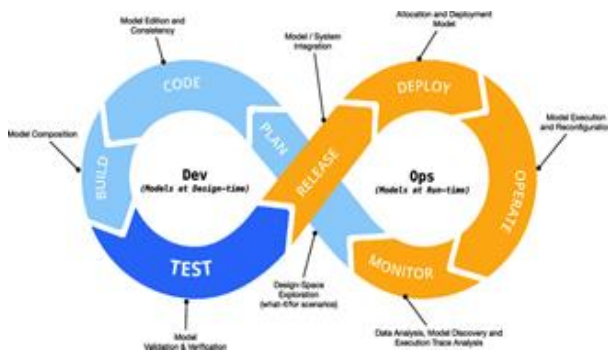


Fig. 2.12. DevOps Model [15]

TABLE I
COMPARATIVE MODELS

MERITS/MODEL	Implementation time	Interface	User Involvement	Simplicity
<i>Waterfall Model</i>	long	minimum	only at beginning	simple
<i>Model with a V-shape</i>	less	minimum	at the beginning	intermediate
<i>Model of iteration</i>	less	crucial	at the beginning	simple
<i>Model with Spirals</i>	depends on project	crucial	at the beginning	intermediate
<i>Big Bang Model</i>	depends on project		little preparation	complex
<i>Modelling Agility</i>	depends on project	model-driven	only at beginning	simple
<i>Prototype Model</i>	less	crucial	at the beginning	simple
<i>Scrum Model</i>	depends on project	maximum	at the beginning	simple
<i>Augmented-reality interactive technology (ARIT)</i>	depends on project	maximum	only at beginning	simple
<i>ADDIE</i>	high	maximum	only at beginning	intermediate
<i>RAD Model</i>	high	minimum	only at beginning	very simple
<i>DevOps Model</i>	high	maximum	only at beginning	simple

MERITS/MODEL	Success rate	Cost	Flexibility	Risk Analysis
<i>Waterfall Model</i>	low	low	rigged	high
<i>Model with a V-shape</i>	high	very high	little flexible	low
<i>Model of iteration</i>	high	high	less flexibility	low
<i>Model with Spirals</i>	high	high	flexible	yes
<i>Big Bang Model</i>	low	low	low	high
<i>Modelling Agility</i>	very high	low	high	low
<i>Prototype Model</i>	good	high	low	low
<i>Scrum Model</i>	good	high	high	low
<i>Augmented-reality interactive technology (ARIT)</i>	good	low	high	low
<i>ADDIE</i>	high	high	high	intermediate
<i>RAD Model</i>	low	low	high	low
<i>DevOps Model</i>	high	low	high	low

International Journal of Emerging Technology and Advanced Engineering

Website: www.ijetae.com (E-ISSN 2250-2459, Scopus Indexed, ISO 9001:2008 Certified Journal, Volume 12, Issue 04, April 2022)

III. CONCLUSION

Various *Modelling Agility*, RAD, and Waterfall are examples of SDLC models., are employed in different relying on businesses on their circumstances. The V-model, for example, provides verification and validation for an organization and is extremely valuable. Each of these software development strategies has its own set of benefits and drawbacks. Nonetheless, the fusion of all these approaches is implemented in today's commercial software development environment. In software development, timing is extremely important. If there is a delay in development, A competitor might swoop in and seize the market. Furthermore, delivering a "bug-filled" In a short period of time, you'll have a finished product may hurt the company's image. Product quality and development time should be traded off. Customers aren't looking for a bug-free product, but they do want one that is simple to use and effective.

REFERENCES

- [1] T. Saravanan, S. Jha, G. Sabharwal, and S. Narayan, "Comparative Analysis of Software Life Cycle Models," Proc. - IEEE 2020 2nd Int. Conf. Adv. Comput. Commun. Control Networking, ICACCCN 2020, pp. 906–909, 2020, doi: 10.1109/ICACCCN51052.2020.9362931.
- [2] M. A. Rather and V. Bhatnagar, "A comparative study of sdlc model," vol. 4, no. 10, pp. 23–29, 2015.
- [3] J. E. T. Akinsola, A. S. Ogunbanwo, O. J. Okesola, I. J. Odun-Ayo, F. D. Ayegbusi, and A. A. Adebiyi, "Comparative Analysis of Software Development Life Cycle Models (SDLC)," Adv. Intell. Syst. Comput., vol. 1224 AISC, no. August, pp. 310–322, 2020, doi: 10.1007/978-3-030-51965-0_27.
- [4] T. Chittagong and T. Islam, "Introducing a New Sdlc Trigon Model for," Proc. Int. Conf. Sustain. Dev. Technol. 4th Ind. Revolut. 2021, no. August 2021, 2021, doi: 10.13140/RG.2.2.21664.10240.
- [5] E. Educba 2020, "Big Bang Model | What is the Big Bang Model with Pros and Cons?" <https://www.educba.com/big-bang-model/> (accessed Jan. 09, 2022).
- [6] K. Dikert, M. Paasivaara, and C. Lassenius, "Challenges and success factors for large-scale agile transformations: A systematic literature review," J. Syst. Softw., vol. 119, no. June, pp. 87–108, 2016, doi: 10.1016/j.jss.2016.06.013.
- [7] T. Grzegorzczak, R. Sliwinski, and J. Kaczmarek, "Attractiveness of augmented reality to consumers," Technol. Anal. Strateg. Manag., vol. 31, no. 11, 2019, doi: 10.1080/09537325.2019.1603368.
- [8] R. Ganpatrao Sabale, "Comparative Study of Prototype Model For Software Engineering With System Development Life Cycle," IOSR J. Eng., vol. 02, no. 07, pp. 21–24, 2012, doi: 10.9790/3021-02722124.
- [9] R. R. Raval and H. M. Rathod, "Comparative Study of Various Process Model in Software Development," Int. J. Comput. Appl., vol. 82, no. 18, pp. 16–19, 2013, doi: 10.5120/14263-2363.
- [10] R. Jayasudha, V. Viswanathan, and P. Shanthi, "Implementation of reuse in the agile software development process scrum," Asian J. Pharm. Clin. Res., vol. 10, no. July 2017, pp. 143–147, 2017, doi: 10.22159/ajpcr.2017.v10s1.19597.
- [11] A. Mujumdar, G. Masiwal, and P. M. Chawan, "Analysis of various Software Process Models," no. June 2012, 2017, [Online]. Available: http://www.researchgate.net/profile/Pramila_Chawan/publication/267427007_Analysis_of_various_Software_Process_Models/links/54f0aa150cf2f9e34efd0776.pdf.
- [12] T. L. Huang and F. H. Liu, "Formation of augmented-reality interactive technology's persuasive effects from the perspective of experiential value," Internet Res., vol. 24, no. 1, pp. 82–109, 2014, doi: 10.1108/IntR-07-2012-0133.
- [13] E. Widyastuti and Susiana, "Using the ADDIE model to develop learning material for actuarial mathematics," J. Phys. Conf. Ser., vol. 1188, no. 1, 2019, doi: 10.1088/1742-6596/1188/1/012052.
- [14] S. Kosasi and I. D. A. Eka Yuliani, "Penerapan Rapid Application Development Pada Sistem Penjualan Sepeda Online," Simetris J. Tek. Mesin, Elektro dan Ilmu Komput., vol. 6, no. 1, p. 27, 2015, doi: 10.24176/simet.v6i1.234.
- [15] M. Kassab and M. Mazzara, Towards Designing Smart Learning Environments with IoT, vol. 12055 LNCS. 2020.